

Table of Contents

BCA 5 th Sem DOT NET: Conceptual Programs with Short Notes:	3
Introduction:	3
Constructor:	3
a) Default/Parameter less Constructor:	3
b) Parameterized Constructor:	4
c) Copy Constructor:	4
d) Static Constructor:.....	5
Inheritance:	6
a) Single Inheritance:	7
b) Multilevel Inheritance:	7
c) Hierarchical Inheritance:	9
d) Multiple Inheritance:	9
Polymorphism:	10
a) Method Overloading:.....	10
b) Method Overriding:	11
Operator Overloading:	12
a) Unary Operator Overloading:	12
b) Binary Operator Overloading:	13
Abstract Method:	14
Abstract Class:	14
Interface:.....	16
a) Multiple Inheritance through interface:	16
Structure:	17
Enumeration (Enum) :	17
Properties:	18
Indexers:	20
Delegates:	21
Multicast Delegates:	23
Anonymous Methods:	24
Lambda Expressions:	24
Generic Delegates:	25

a) Func :.....	25
b) Action:	25
c) Predicate:	25
Exception Handling:	26
LINQ to SQL:	28
#TU (2020) 20.a) Write a program to select employees whose salary is greater than 20000 and whose address is Kathmandu using LINQ	29
ASP.NET:	33
Example 1: Creating a Login Form which directs to “success” page if correct input and “failure” page if incorrect input in different page after submitting the credentials.....	33
#TU (2020) 18.a), TU (2021) 20 : Write a program to create user/student registration form in one ASP.NET page and display filled data in another page.	37
#TU (2022) 17. Write a program to create form for calculating simple interest in one ASP.NET page and display the simple interest in another page of ASP.NET.	40
# TU (2020) 18 b) : Write a program for handling exception in ASP.NET	43

BCA 5th Sem DOT NET: Conceptual Programs with Short Notes:

Introduction:

- ➔ The problem with older programming languages is that we cannot use the same language for developing all kinds of applications.

Types of Applications:

- a) **Desktop Applications** (C, C++, Visual Basic)
 - b) **Web Applications** (PHP, CGI, ASP, JavaScript)
 - c) **Mobile Applications**
- ➔ The problem is we have to learn multiple programming language by which there are confusions in syntaxes and all.
 - ➔ For addressing this issue, we use DOT NET.
 - ➔ DOT NET is a product provided by Microsoft for developing any kinds of applications that can include any of desktop, web or mobile applications under a single roof.
 - ➔ C# is a most widely used and popular language in .NET which is an extension of C++, used to develop any kind of application not only desktop application as in C++.
 - ➔ Visual Studio is an IDE by Microsoft for developing .NET application primarily used for developing console, windows and web applications.

Constructor:

- ➔ It is a special method present under a class responsible for initializing the variables of that class.
- ➔ The name of a constructor method is exactly the same name of the class in which it was present.
- ➔ Moreover, it's a non-value returning method in which each and every class requires it if we want to create the instance of that class.

Types of Constructor:

a) Default/Parameter less Constructor:

- ➔ If a constructor method doesn't take any parameters then we call that as default or parameter less constructor:
- ➔ These constructors can be defined by a programmer explicitly or else will be default implicitly, provided there is no explicit constructor under the class.

```
class defaultCons
{
    public defaultCons()
    {
        Console.WriteLine("Constructor is called");
    }

    static void Main()
```

```

    {
        defaultCons obj=new defaultCons();
        Console.ReadLine();
    }
}

```

b) Parameterized Constructor:

➔ If a constructor method is defined with any parameters, we call that as parameterized constructor.

➔ These constructors can be defined by the programmers only.

```

class parameterized
{
    public parameterized(int i)
    {
        Console.WriteLine("The value is:" + i);
    }
    static void Main()
    {
        parameterized cd1 = new parameterized(10);
        parameterized cd2 = new parameterized(20);
        Console.ReadLine();
    }
}

```

Output:

The value is:10

The value is:20

c) Copy Constructor:

➔ If we want to create multiple instances with the same values, then we use these copy constructors.

➔ In a copy constructor, the constructor takes the same class as a parameter to it.

```

class copyCon
{
    int x;
    public copyCon(int i) //Parameterized
    {
        x = i;
    }
    public copyCon(copyCon obj) //Copy
    {
        x = obj.x;
    }
    public void disp()
    {
        Console.WriteLine("Value of x is: " + x);
    }
}

```

```

static void Main()
{
    copyCon cd1 = new copyCon(10);
    cd1.disp();
    copyCon cd2 = new copyCon(cd1);
    cd2.disp();
    Console.ReadLine();
}
}

```

Output:

Value of x is: 10

Value of x is: 10

d) Static Constructor:

- ➔ If a constructor is declared by using static modifier, we call that as static constructor.
- ➔ These constructors are first to execute under any class.
- ➔ Static constructors can't be parameterized, so overloading it is not possible.

```

class staticCon
{
    static void Main()
    {
        Console.WriteLine("Main method is executed");
    }
    static staticCon()
    {
        Console.WriteLine("Static constructor is executed");
    }
}

```

Output:

Static constructor is executed

Main method is executed

Inheritance:

- ➔ It's a mechanism of consuming the members of one class in another class by establishing parent/child relationship between classes.
- ➔ It ignores re-writing the code which affects the size of application and rather provides code re-usability.

Conceptual Example:

class1.cs

```
using System;
namespace inheritance
{
    class class1
    {
        public void test1()
        {
            Console.WriteLine("Method 1");
        }
        public void test2()
        {
            Console.WriteLine("Method 2");
        }
    }
}
```

class2.cs

```
using System;
namespace inheritance
{
    class class2 : class1
    {
        public void test3()
        {
            Console.WriteLine("Method 3");
        }
        static void Main()
        {
            class2 c = new class2();
            c.test1();
            c.test2();
            c.test3();
            Console.ReadLine();
        }
    }
}
```

Types of Inheritance:

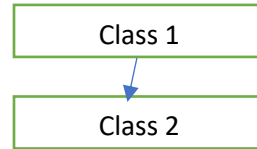
a) Single Inheritance:

Class1.cs

```
using System;
namespace inheritance
{
    class class1
    {
        public int a = 10, b = 5;
    }
}
```

Class2.cs

```
using System;
namespace inheritance
{
    class class2 : class1
    {
        public void test()
        {
            Console.WriteLine("Value of a is: "+a);
            Console.WriteLine("Value of b is: " + b);
        }
        static void Main(string[] args)
        {
            class2 c = new class2();
            c.test();
            Console.ReadLine();
        }
    }
}
```



Output:

Value of a is: 10

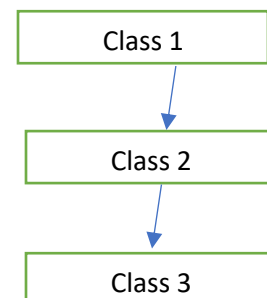
Value of b is: 5

b) Multilevel Inheritance:

class1.cs

```
using System;
namespace inheritance
{
    class class1
    {
        public int a, b, c;
        public void readData(int a, int b)
        {

```



```

        this.a = a;
        this.b = b;
    }
    public void Display()
    {
        Console.WriteLine("The value of a is: " + a);
        Console.WriteLine("The value of b is: " + b);
    }
}
}

```

class2.cs

```

class class2 : class1
{
    public void add()
    {
        base.c = base.a + base.b;
        Console.WriteLine("Sum: " + base.c);
    }
}

```

class3.cs

```

class class3 : class2
{
    public void sub()
    {
        base.c = base.a - base.b;
        Console.WriteLine("Difference: " + base.c);
    }
    static void Main()
    {
        class3 obj = new class3();
        obj.readData(10, 7);
        obj.Display();
        obj.add();
        obj.sub();
        Console.ReadLine();
    }
}

```

Output:

The value of a is: 10

The value of b is: 7

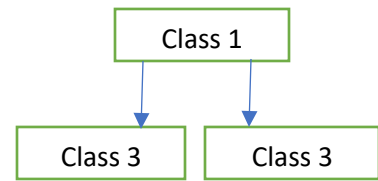
Sum: 17

Difference: 3

c) Hierarchical Inheritance:

class1.cs

```
class class1
{
    public int a, b;
    public void readData(int a, int b)
    {
        this.a = a;
        this.b = b;
    }
}
```



class2.cs

```
class class2 : class1
{
    public void areaRec()
    {
        base.readData(2, 2);
        int area = base.a * base.b;
        Console.WriteLine("Area of rectangle= " + area);
    }
}
```

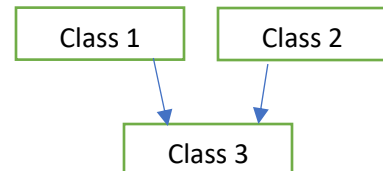
class3.cs

```
class class3 : class1
{
    public void areaTri()
    {
        base.readData(2, 2);
        double area = 0.5 * base.a * base.b;
        Console.WriteLine("Area of triangle= " + area);
    }
    static void Main()
    {
        class3 tri = new class3();
        tri.areaTri();
        class2 rec = new class2();
        rec.areaRec();
        Console.ReadLine();
    }
}
```

Output:

Area of triangle= 2

Area of rectangle= 4



d) Multiple Inheritance:

- ➔ Multiple inheritance is not supported through classes so we need interface to implement it. You can understand the concept by referring to **interface** topic somewhere below.

Polymorphism:

- ➔ It is a concept by which we can perform a single action in different ways.
- ➔ 2 Greek words 'poly'-many and 'morphs'-forms.

Types of polymorphism:

a) Method Overloading:

is a feature that allows a class to have more than one method having the same name, if their argument lists are different.

Example:

```
using System;
namespace methodOverloading
{
    class elliot
    {
        public void sum(int a, int b)
        {
            int s = a + b;
            Console.WriteLine("Sum of two integer no is: " + s);
        }
        public void sum(double a, double b)
        {
            double d = a + b;
            Console.WriteLine("Sum of two double no is: " + d);
        }
        public void sum(int a, int b, int c)
        {
            int t = a + b + c;
            Console.WriteLine("Sum of three integer no is: " + t);
        }
    }
    class alderson
    {
        static void Main(string[] args)
        {
            elliot fs = new elliot();
            fs.sum(10, 20);
            fs.sum(7.77, 10.77);
            fs.sum(7, 10, 11);
        }
    }
}
```

Output:

```
Sum of two integer no is: 30
Sum of two double no is: 18.54
Sum of three integer no is: 28
```

b) Method Overriding:

➔ Its an approach of re-implementing the method of the parent class under the child class with the same signature.

➔ **Example:**

subject.cs

```
using System;
namespace methodOverriding
{
    class subject
    {
        public virtual void study()
        {
            Console.WriteLine("Study all the subjects");
        }
    }
}
```

Dotnet.cs

```
class dotnet : subject
{
    public override void study()
    {
        base.study();
        Console.WriteLine("Study DOT NET subject");
    }
    static void Main(string[] args)
    {
        dotnet obj = new dotnet();
        obj.study();
        Console.ReadLine();
    }
}
```

Output:

Study all the subjects
Study DOT NET subject

Note: If we want to override a parent's method under the child class, firstly that method should be declared by using the virtual modifier in parent class.

Class1

```
public virtual void Test()    //overridable
```

Any virtual method of the parent class can be overridden by the child if required by using the override modifier.

Class2

```
public override void Test()    //overriding
```

Operator Overloading:

- ➔ It's a way of re-defining the c# operators.
- ➔ It gives the ability to use the same operator to do various operations.
- ➔ Operator overloading is nothing but a mechanism for re-defining in-built operations without changing its own meaning.

2 Types:

a) Unary Operator Overloading:

- ➔ If you are able to perform the operation with one operand.

Source Code:

```
using System;
namespace unaryOverload
{
    class calculator
    {
        public int num1, num2;
        public calculator(int n1, int n2)
        {
            num1 = n1;
            num2 = n2;
        }
        public static calculator operator -(calculator c)
        {
            c.num1 = -c.num1;
            c.num2 = -c.num2;
            return c;
        }
        public void display()
        {
            Console.WriteLine("Numer1 = " + num1);
            Console.WriteLine("Numer2 = " + num2);
        }
        static void Main(string[] args)
        {
            calculator calc = new calculator(7,-10);
            calc = -calc;
            calc.display();
            Console.ReadLine();
        }
    }
}
```

Output:

```
Numer1 = -7
Numer2 = 10
```

b) Binary Operator Overloading:

➔ If you are going to perform the operation with two operands.

Source Code:

```
using System;
namespace binaryOverload
{
    class calculator
    {
        public int n=0;
        public calculator() {}
        public calculator(int num)
        {
            n = num;
        }
        public static calculator operator +(calculator calc1, calculator calc2)
        {
            calculator calc3 = new calculator(0);
            calc3.n = calc1.n + calc2.n;
            return calc3;
        }
        public void display()
        {
            Console.WriteLine("{0}", n);
        }
    }
    class entryPoint
    {
        static void Main(string[] args)
        {
            calculator number1 = new calculator(250);
            calculator number2 = new calculator(40);
            calculator number3 = new calculator();
            number3 = number1 + number2;
            number1.display();
            number2.display();
            number3.display();
        }
    }
}
```

Output:

```
250
40
290
```

Abstract Method:

- ➔ A method without any method body is known as an abstract method.

//Non – abstract method:

```
public void Add (int x, int y)
{
}
```

//Abstract Method:

```
public abstract void Add (int x, int y);
```

- ➔ If a method is declared as an abstract under any class, then the child class of that class is responsible for implementing the method without fail.

Abstract Class:

- ➔ A class under which we define abstract methods is known as an abstract class.
- ➔ This class has abstract methods but can contain non-abstract methods also.
- ➔ The concept is similar to method overriding but that was optional and it is mandatory to implement that method under the child class.

```
abstract class class1
{
    public abstract void show();
}
class2 : class1
{
    public override void show()    //Mandatory
    {
        -Implementation
    }
}
```

parent.cs

```
using System;
namespace abstractClass
{
    abstract class parent
    {
        public void add(int x, int y)
        {
            Console.WriteLine(x + y);
        }
        public void sub(int x, int y)
        {
            Console.WriteLine(x - y);
        }
        public abstract void mul(int x, int y);
        public abstract void div(int x, int y);
    }
}
```

child.cs

```
using System;
namespace abstractClass
{
    class child : parent
    {
        public override void mul(int x, int y)
        {
            Console.WriteLine(x * y);
        }
        public override void div(int x, int y)
        {
            Console.WriteLine(x / y);
        }
    }
}
```

entryPoint.cs

```
using System;
namespace abstractClass
{
    class entryPoint
    {
        static void Main(string[] args)
        {
            child c = new child();
            parent p = c; //Reference of parent class since overridden
            p.add(7, 7);
            p.sub(7, 10);
            p.mul(7, 11);
            p.div(77, 7);
            Console.ReadLine();
        }
    }
}
```

Output:

```
14
-3
77
11
```

Interface:

- ➔ An interface is also an user-defined type like a class, but can only contain abstract methods in it.
- ➔ In class, child class inherits for consuming members of parent whereas child class inherits from interface to implement it.
- ➔ The **default scope** of the members of an **interface** is **public** whereas its private in case of interface.
- ➔ By default, every member of an interface is abstract so we don't require to write it
public abstract void Add(int a, int b); //In class
void Add(int a, int b); //In interface
- ➔ We **can't** declare any fields/variables under an interface like **int x**;
- ➔ **No need** to write **override** in child class.

a) Multiple Inheritance through interface:

```
using System;
{
    interface Interface1
    {
        void test();
        void show();
    }
    interface Interface2
    {
        void test();
        void show();
    }
    class childClass : Interface1, Interface2
    {
        public void test()
        {
            Console.WriteLine("Interface's method is now implemented in the child class successfully");
        }
        void Interface1.show()
        {
            Console.WriteLine("Declared in Interface1 and implemented in class");
        }
        void Interface2.show()
        {
            Console.WriteLine("Declared in Interface2 and implemented in class");
        }
        static void Main(string[] args)
        {
            childClass obj = new childClass();
            obj.test();

            Interface1 i1 = obj;
            i1.show();
            Interface2 i2 = obj;
            i2.show();
            Console.ReadLine();
        }
    }
}
```

Output:

Interface's method is now implemented in the child class successfully

Declared in Interface1 and implemented in class

Declared in Interface2 and implemented in class

Structure:

SN	Class	Structure
1	Reference type	Value type
2	Memory allocation for instances of class is performed on managed heap	Memory allocation for instances of structure is performed on stack
3	We use class for entities with larger volumes of data	We use struct for entities with smaller volumes of data
4	'new' is mandatory for creating the instances	Optional
5	Fields of a class can be initialized at the time of declaration	Its not possible with fields in structure
6	Class can be inherited by other classes	Structure can't be inherited by other structures i.e. it doesn't support inheritance

Enumeration (Enum) :

- ➔ A set of named constant values.
- ➔ An enum is an user-defined type, so it is always better to define an enum under the namespace, but it is also possible to define it under the class or structure.

Source Code:

```
using System;
namespace enumeration
{
    public enum Days
    {
        Sunday=1, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday
    }
    class Class1
    {
        static void Main(string[] args)
        {
            foreach (int i in Enum.GetValues(typeof(Days)))
                Console.WriteLine(i + ":" + (Days)i);
            Console.ReadLine();
        }
    }
}
```

Output:

- 1:Sunday
- 2:Monday
- 3:Tuesday
- 4:Wednesday
- 5:Thursday
- 6:Friday
- 7:Saturday

Properties:

- ➔ Properties are members of the class using which we can expose the values associated with a class to the outside environment.
- ➔ They use the accessors, **get & set** through which the values of the private fields can be read, written or manipulated.
- ➔ This is a good programming practice since the data fields aren't directly accessible to us, so we must use the get/set methods to access the data fields.

Source Code:

customer.cs

```
using System;
namespace properties
{
    class customer
    {
        int _cid;
        string _cname;
        public customer(int cid, string cname)
        {
            _cid = cid;
            _cname = cname;
        }
        public int cid
        {
            get { return _cid; }
        }
        public string cname
        {
            get { return _cname; }
            set { _cname = value; }
        }
    }
}
```

testCustome.cs

```
using System;
namespace properties
{
    class testCustomer
    {
        static void Main(string[] args)
        {
            customer obj = new customer(101, "Diwas");
            Console.WriteLine("Customer ID: " + obj.cid);

            Console.WriteLine("Customer Name: " + obj.cname);
            obj.cname += "Lamichhane";
            Console.WriteLine("Modified Customer Name: " + obj.cname);
            Console.ReadLine();

        }
    }
}
```

Output:

Customer ID: 101

Customer Name: Diwas

Modified Customer Name: DiwasLamichhane

Indexers:

➔ Indexer is also something like a property, but provides access to the values using the index positions.

➔ Syntax:

modifier type this [parameter list]

```
{  
    get {statements;}  
    set {statements;}  
}
```

➔ Example:

```
String s="Hello";  
Console.WriteLine(s[0]);    //H  
Console.WriteLine(s[3]);    //l
```

Source Code:

```
using System;  
namespace indexerProject  
{  
    class employee  
    {  
        int eno;  
        string ename;  
        double salary;  
        public employee(int eno, string ename, double salary)  
        {  
            this.eno = eno;  
            this.ename = ename;  
            this.salary = salary;  
        }  
        public object this[int index]  
        {  
            get  
            {  
                if (index == 0)  
                    return eno;  
                else if (index == 1)  
                    return ename;  
                else if (index == 2)  
                    return salary;  
                return null;  
            }  
            set  
            {  
                if (index == 0)  
                    eno = (int)value;  
                else if (index == 1)  
                    ename = (string)value;  
                else if (index == 2)  
                    salary = (double)value;  
            }  
        }  
    }  
}
```

```

class testEmployee
{
    static void Main(string[] args)
    {
        employee emp = new employee(74292, "Diwas Lama", 70500.00);
        Console.WriteLine("Employee No: " + emp[0]);
        Console.WriteLine("Employee Name: " + emp[1]);
        Console.WriteLine("Employee Salary: " + emp[2]);

        emp[1] = "Diwas Lamichhane";
        emp[2] = 75500.00;

        Console.WriteLine();
        Console.WriteLine("Employee No: " + emp[0]);
        Console.WriteLine("Employee Name: " + emp[1]);
        Console.WriteLine("Employee Salary: " + emp[2]);
        Console.ReadLine();
    }
}

```

Output:

Employee No: 74292

Employee Name: Diwas Lama

Employee Salary: 70500

Employee No: 74292

Employee Name: Diwas Lamichhane

Employee Salary: 75500

Delegates:

- ➔ Delegate is an object that knows how to call a method.
- ➔ In C#, we can call methods in 2 different ways:
 - 1) By using the instance of a class if it is a non-static method and name of the class if it is a static method.
 - 2) By using the delegate which can call both static and non-static methods.
- ➔ To call a method by using delegates, we have 3 steps:
 - Step 1: Defining a delegate:


```

          Public void addNums (int a, int b)           //NORMAL
          Public delegate void addDelegates (int a, int b); //DELEGATE
          
```
 - Step 2: Instantiating the delegate:


```

          addDelegates ad=new addDelegates (p.addNums);
          
```
 - Step 3: Calling the delegate:


```

          ad(77,100);
          
```

Source Code:

```
using System;
namespace delegates
{
    public delegate void addDelegate(int a, int b);
    public delegate string sayDelegate(string str);
    class _12bajyo
    {
        public void addNums(int a, int b)
        {
            Console.WriteLine(a + b);
        }
        public static string sayHello(string name)
        {
            return "Hello" + name;
        }
        static void Main(string[] args)
        {
            _12bajyo p = new _12bajyo();
            addDelegate ad = new addDelegate(p.addNums);
            ad.Invoke(77, 100);

            sayDelegate sd = new sayDelegate(_12bajyo.sayHello);
            string str = sd("Diwas");
            Console.WriteLine(str);
            Console.ReadLine();
        }
    }
}
```

Output:

177

HelloDiwas

Multicast Delegates:

➔ A delegate holds the reference of a method whereas a multicast delegate holds the reference of more than one method.

Example:

```
using System;
namespace multicastDelegates
{
    public delegate void rectDelegate(int width, int height);
    class rectangle
    {
        public void getArea(int width, int height)
        {
            Console.WriteLine("Area of rectangle= " + width * height);
        }
        public void getPerimeter(int width, int height)
        {
            Console.WriteLine("Perimeter of rectangle= " + 2 * (width + height));
        }
        static void Main(string[] args)
        {
            rectangle rect = new rectangle();
            rectDelegate obj = rect.getArea;
            obj += rect.getPerimeter;

            obj(2, 2);
            Console.WriteLine();
            obj(7, 10);
            Console.ReadLine();
        }
    }
}
```

Output:

Area of rectangle= 4

Perimeter of rectangle= 8

Area of rectangle= 70

Perimeter of rectangle= 34

Anonymous Methods:

- ➔ It says, without binding a named method to the delegate, we can bind an unnamed code block to the delegate.
- ➔ Here, we don't have to look elsewhere for the logic because the logic is directly implemented at the place where the binding has been performed.
- ➔ This gives lesser typing work by using the 'delegate' keyword.

Example:

```
using System;
namespace anonymousMethods
{
    public delegate string greetingsDelegate(string name);
    class greetings
    {
        static void Main(string[] args)
        {
            greetingsDelegate obj = delegate (string name)
            {
                return "Hello" + name + " what up?";
            };
            string str = obj.Invoke("biatch!");
            Console.WriteLine(str);
            Console.ReadLine();
        }
    }
}
```

Output:

Hellobiatch! what up?

Lambda Expressions:

- ➔ It is a shorthand for anonymous methods.
- ➔ Here, we don't need to use either of 'delegate' keyword or parameter types.
- ➔ The '=>' is the lambda operator which is used for all lambda expressions.

Example:

```
using System;
namespace lambdaExpressions
{
    public delegate string greetingsDelegate(string name);
    class lambdaExample
    {
        static void Main(string[] args)
        {
            greetingsDelegate obj = (name) =>
            {
                return "Hello " + name + " what up?";
            };
        }
    }
}
```



```

    };
    string str = obj.Invoke("biatch!");
    Console.WriteLine(str);
    Console.ReadLine();
}
}
}

```

Output:

Hello biatch! what up?

Generic Delegates:

a) Func :

is used when your method is going to return a value (double, int, etc.)

For example,

```
public static double addNums(int x, float y, double z)
```

b) Action:

is used when your method is going to be a non-value returning method (void, etc.)

For example,

```
public static void addNums(int x, float y, double z)
```

c) Predicate:

is used when we want the return type as a Boolean which takes only a single parameter.

For example,

```
public static bool checkLength(string str)
```

Example:

```

using System;
namespace genericDelegates
{
    class generic
    {
        public static double addNums1(int x, float y, double z)
        {
            return x + y + z;
        }
        public static void addNums2(int x, float y, double z)
        {
            Console.WriteLine(x + y + z);
        }
        public static bool checkLength(string str)
        {
            if (str.Length > 5)
                return true;
            return false;
        }
    }
}

```

```

    }
    static void Main()
    {
        //func ma first ko 3ta input parameter and 4th ko double vaneko output parameter
        Func<int, float, double, double> obj1 = addNums1;
        double result = obj1.Invoke(7, 10.7f, 11.77);
        Console.WriteLine(result);

        //action ma output parameter void vako le output ko type dina pardaina
        Action<int, float, double> obj2 = addNums2;
        obj2.Invoke(7, 10.7f, 11.77);

        Predicate<string> obj3 = checkLength;
        bool status = obj3.Invoke("Diwas dai");
        Console.WriteLine(status);

        Console.ReadLine();
    }
}

```

Output:

```

29.470000762939453
29.470000762939453
True

```

Exception Handling:

- ➔ Whenever, a run-time error occurs in your code, the program terminates abnormally on the same line where the error got occurred without executing the next lines of code. So, its dangerous.
- ➔ An exception is a problem that arises during the execution of a program.
- ➔ C# exceptions are represented by classes which comes into play whenever a run time error occurs in the code for abnormally terminating your program.

Exception Classes:

- **IndexOutOfRangeException** : printing values more than no. of values in an array
- **DivideByZeroException** : if you try to divide a number by zero
- **OverflowException** : a value exceeding the size limit
- **FormatException** : invalid type conversion (eg. converting double into int)
- **NullReferenceException** : handles errors generated from referencing a null object
- **OutOfMemoryException**
- **StackOverflowException**

- ➔ Exception handling is the process of stopping the abnormal termination of the program whenever a run time error is occurring inside the program.
- ➔ C# exception handling is built upon four keywords:
 - try : All statements will execute only if there is no exception
 - catch : All statements will execute only if there is an exception
 - finally : All statements will execute in both scenarios
 - throw : Exception not defined by CLR but by application developer

Implementation Example:

```
class exClasses
{
    static void Main()
    {
        try
        {
            Console.WriteLine("Enter 1st number:");
            int x = int.Parse(Console.ReadLine());
            Console.WriteLine("Enter 2nd number:");
            int y = int.Parse(Console.ReadLine());
            if (y == 1)
            {
                return;
            }
            if (y % 2 > 0)
            {
                throw new ApplicationException("Divisor value cannot be odd
number");
            }
            int z = x / y;
            Console.WriteLine("The result is:" + z);
        }
        catch (DivideByZeroException ex1)
        {
            Console.WriteLine(ex1.Message);
        }
        catch (FormatException ex2)
        {
            Console.WriteLine("Input must be numeric");
        }
        catch (Exception ex)
        {
            Console.WriteLine(ex.Message);
        }
        finally
        {
            Console.WriteLine("End of the program");
        }
    }
}
```

Output:

Enter 1st number:

77

Enter 2nd number:

7

Divisor value cannot be odd number

End of the program

LINQ to SQL:

- ➔ Using LINQ, we can write queries on a wide variety of data sources like Arrays, Collections, Database Tables, Datasets, XML Data, etc.

Example 1: LINQ code for sorting of numbers greater than 40

```
using System;
using System.Linq;

namespace linqDemo
{
    class Class1
    {
        static void Main()
        {
            int[] arr = { 7, 77, 10, 11, 45, 67, 88, 34, 23, 39, 68, 79, 55, 27, 97, 74, 8, 12 };
            var brr = from i in arr where i > 40 orderby i descending select i;

            foreach (int x in brr)
                Console.WriteLine(x + " ");
        }
    }
}
```

Output:

97 88 79 77 74 68 67 55 45

Logic for this:

- ➔ var brr = from i in arr select i; //fetches all data from array
- ➔ var brr = from i in arr where i>40 select i; //fetches data of values greater than 40
- ➔ var brr = from i in arr where i>40 orderby i select i; //in sorted (ascending) order
- ➔ var brr = from i in arr where i>40 orderby i descending select i; //in descending order

#TU (2020) 20.a) Write a program to select employees whose salary is greater than 20000 and whose address is Kathmandu using LINQ

Note:

- ➔ *This can be taken as a practical implementation, for alternate way (for written exams) you can take reference from below this example!*
- ➔ I created the database table **Employees** in **SQL Server Management Studio(SSML)** directly.
- ➔ Then I used **LINQ to SQL** class in Visual Studio by adding **DataClasses1.dbml**
- ➔ I configured the database from **Server Explorer** and dragged to the designer surface.
- ➔ If you want the **SQL query** codes are here which I assume is optional:

1. SQL Table(created in SQL Server Management Studio):

```
CREATE TABLE Employees (  
    Id INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Salary FLOAT  
    Address VARCHAR(100)  
);
```

```
INSERT INTO Employees VALUES  
(1, 'Diwas', 25000, 'Kathmandu'),  
(.....);
```

2. Windows Form Code (Form1.cs)

```
private void btnFilter_Click(object sender, EventArgs e)  
{  
    DataClasses1DataContext db = new DataClasses1DataContext();  
    var result = from emp in db.Employees  
                 where emp.Salary > 20000  
                 && emp.Address == "Kathmandu"  
                 select emp;  
    dataGridView1.DataSource = result.ToList();  
}  
  
private void Form1_Load(object sender, EventArgs e)  
{  
    DataClasses1DataContext db = new DataClasses1DataContext();  
    dataGridView1.DataSource = db.Employees.ToList();  
}
```

Output Previews:

Before Clicking on button:

Form1

Filter Employees

	Id	Name	Salary	Address
▶	1	Diwas	25000	Kathmandu
	2	Nishab	18000	Pokhara
	3	Sandesh	22000	Dhulikhel
	4	Upiya	20005	Kathmandu
	5	Rupak	2	Kathmandu

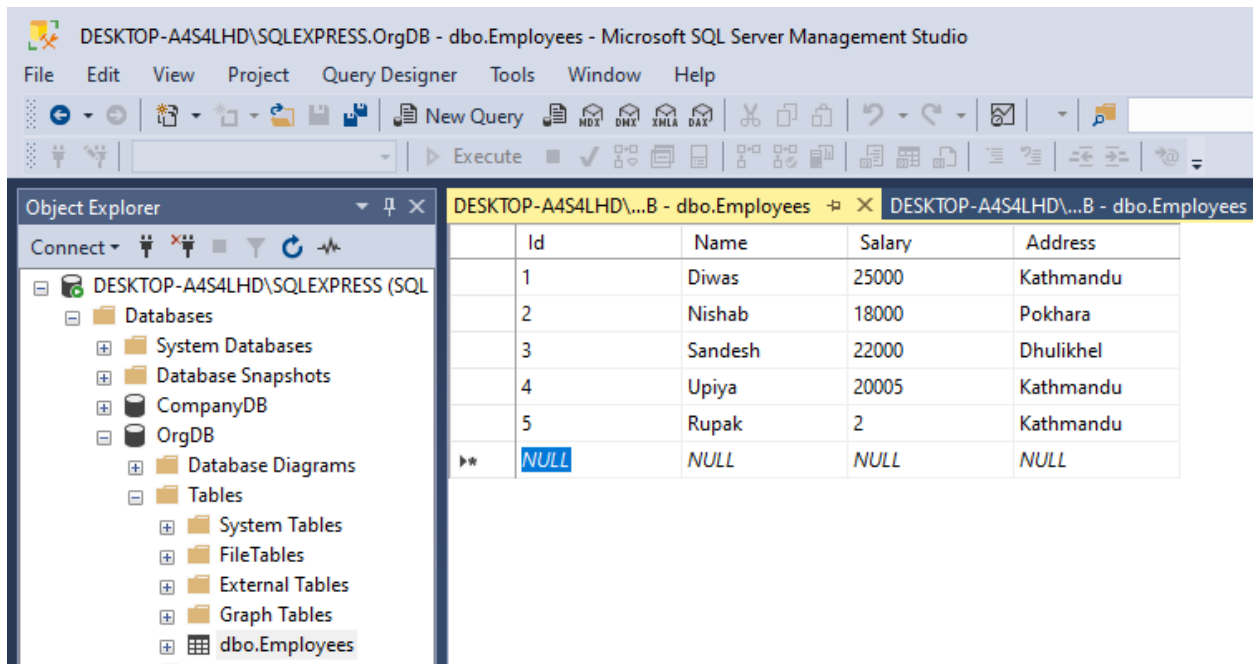
After Clicking the “Filter Employees” button:

Form1

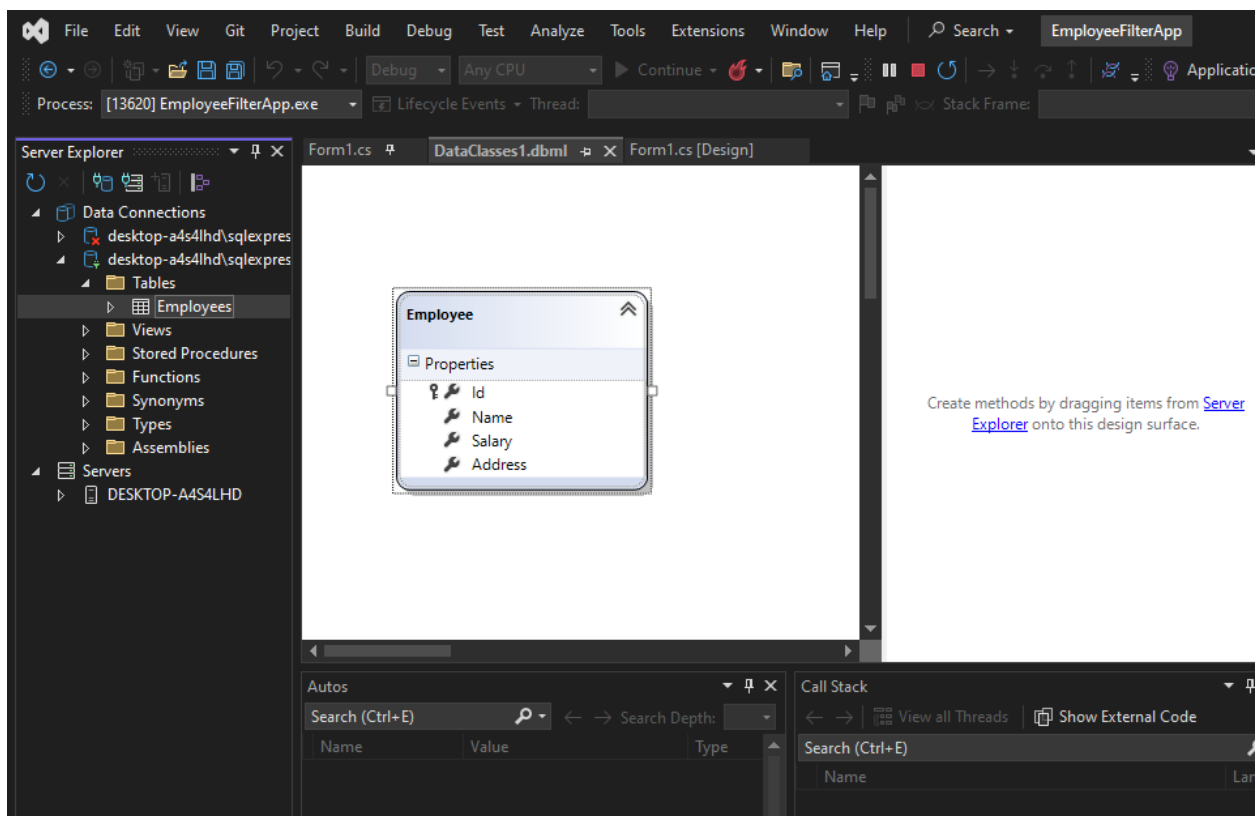
Filter Employees

	Id	Name	Salary	Address
▶	1	Diwas	25000	Kathmandu
	4	Upiya	20005	Kathmandu

Database and Table created from SQL Server Management Studio:



Configured the data connecting the database from the Server Explorer:



Alternate Way (Esp. for Exam POV):

```
using System;
```

```
using System.Linq;
```

```
using System.Collections.Generic;
```

```
public class Employee
```

```
{
```

```
    public string Name { get; set; }
```

```
    public decimal Salary { get; set; }
```

```
    public string Address { get; set; }
```

```
}
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
    {
```

```
        List<Employee> employees = new List<Employee>
```

```
        {
```

```
            new Employee { Name="Diwas", Salary=25000, Address="Kathmandu" },
```

```
            new Employee { Name="Nishab", Salary=22000, Address="Kathmandu" },
```

```
            new Employee { Name="Aakritee", Salary=100000, Address="Chitwan" },
```

```
            new Employee { Name="Sandesh", Salary=66666, Address="Bangkok" },
```

```
            new Employee { Name="Rupak", Salary=2, Address="Kathmandu" }
```

```
        };
```

```
        var selEmployees = from emp in employees
```

```
            where emp.Salary > 20000 && emp.Address == "Kathmandu"
```

```
            select emp;
```

```
        Console.WriteLine("Result=");
```

```
        foreach (var emp in selEmployees)
```



```

    {
        Console.WriteLine($"Name: {emp.Name}, Salary: {emp.Salary}, Address: {emp.Address}");
    }
}
}

```

Output:

```

Result=
Name: Diwas, Salary: 25000, Address: Kathmandu
Name: Nishab, Salary: 22000, Address: Kathmandu

```

ASP.NET:

- ➔ It is an application framework from Microsoft for developing web applications to produce dynamic webpages.
- ➔ ASP stands for Active Server Pages.
- ➔ The coding techniques include Inline Coding and Code Behind techniques.

Example 1: Creating a Login Form which directs to “success” page if correct input and “failure” page if incorrect input in different page after submitting the credentials.

login.aspx:

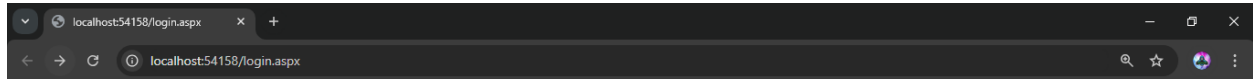
```

<body>
    <form id="form1" runat="server">
        <div>
            <table align="center">
                <caption>Login Form</caption>
                <tr>
                    <td>Enter User Name:</td>
                    <td><asp:TextBox ID="txtName" runat="server" Width="150px"></asp:TextBox></td>
                </tr>
                <tr>
                    <td>Enter Password:</td>
                    <td><asp:TextBox ID="txtPwd" runat="server" TextMode="Password" Width="150px"></asp:TextBox></td>
                </tr>
                <tr>
                    <td align="right"><asp:Button ID="btnLogin" runat="server" Text="Login" OnClick="btnLogin_Click"/></td>
                    <td><asp:Button ID="btnReset" runat="server" Text="Reset" OnClick="btnReset_Click"/></td>
                </tr>
            </table>
        </div>
    </form>
</body>

```

login.aspx.cs:

```
protected void Page_Load(object sender, EventArgs e)
{
    if (!IsPostBack)
        txtName.Focus();
}
protected void btnLogin_Click(object sender, EventArgs e)
{
    if (txtName.Text == "admin" && txtPwd.Text == "admin")
        Server.Transfer("success.aspx");
    else
        Server.Transfer("failure.aspx");
}
protected void btnReset_Click(object sender, EventArgs e)
{
    txtName.Text = txtPwd.Text = "";
    txtName.Focus();
}
```



Login Form

Enter User Name:

Enter Password:



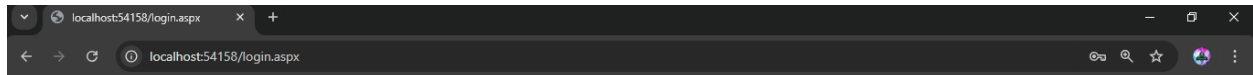
success.aspx:

```
<body>
  <form id="form1" runat="server">
    <div>
      <h1 style="background-color:aqua; color:maroon; text-align:center">
        Login successful!
      </h1>
    </div>
  </form>
</body>
```

success.aspx.cs:

```
protected void Page_Load(object sender, EventArgs e)
{
    Page pp = PreviousPage;
    Control ctrl = pp.FindControl("txtName");
    TextBox tb = (TextBox)ctrl;
    string Name = tb.Text;

    Response.Write("Hello " + Name + ", have a nice day");
}
```



Hello admin, have a nice day

Login successful!



failure.aspx:

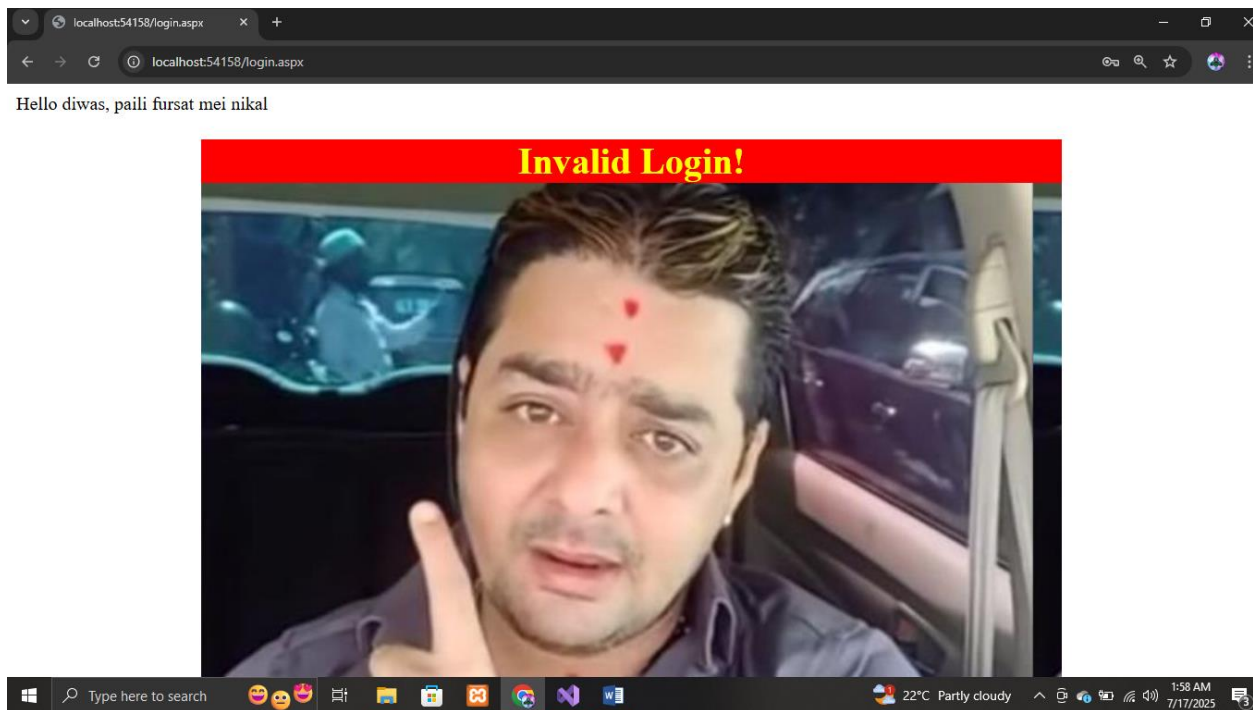
```
<head runat="server">
    <title></title>
    <style>
        .backimg{
            background-image:url(img/failureImg.jpg);
            width:70%;
            height:70%;
        }
    </style>
</head>
<body>
    <form id="form1" runat="server">
        <div align="center" style="height:720px">
            <div class="backimg">
                <h1 style="background-color:red; color:yellow; text-align:center">
                    Invalid Login!
                </h1>
            </div>
        </div>
    </form>
</body>
```

```
</div>
</div>
</form>
</body>
```

failure.aspx.cs:

```
protected void Page_Load(object sender, EventArgs e)
{
    Page pp = PreviousPage;
    Control ctrl = pp.FindControl("txtName");
    TextBox tb = (TextBox)ctrl;
    string Name = tb.Text;

    Response.Write("Hello " + Name + ", paili fursat mei nikal");
}
```



#TU (2020) 18.a), TU (2021) 20 : Write a program to create user/student registration form in one ASP.NET page and display filled data in another page.

register.aspx:

```
<body>
  <form id="form1" runat="server">
    <div>
      <table align="center">
        <caption>Registration Form</caption>
        <tr>
          <td>Enter your first name:</td>
          <td><asp:TextBox ID="txtFname" runat="server" width="150px"></asp:TextBox></td>
        </tr>
        <tr>
          <td>Enter your last name:</td>
          <td><asp:TextBox ID="txtLname" runat="server" Width="150px"></asp:TextBox></td>
        </tr>
        <tr>
          <td>Enter your email id:</td>
          <td><asp:TextBox ID="txtEmail" runat="server" Width="150px"></asp:TextBox></td>
        </tr>
        <tr>
          <td>Enter your gender:</td>
          <td>
            <asp:RadioButtonList ID="rblGender" runat="server">
              <asp:ListItem>Male</asp:ListItem>
              <asp:ListItem>Female</asp:ListItem>
            </asp:RadioButtonList>
          </td>
        </tr>
        <tr>
          <td>Select a course:</td>
          <td>
            <asp:DropDownList ID="ddlCourse" runat="server">
              <asp:ListItem>BCA</asp:ListItem>
              <asp:ListItem>BBA</asp:ListItem>
              <asp:ListItem>BBM</asp:ListItem>
              <asp:ListItem>BHM</asp:ListItem>
              <asp:ListItem>BBS</asp:ListItem>
              <asp:ListItem>MCA</asp:ListItem>
              <asp:ListItem>MBS</asp:ListItem>
            </asp:DropDownList>
          </td>
        </tr>
        <tr>
          <td align="right"><asp:Button ID="btnSubmit" runat="server" Text="Register"
            OnClick="btnSubmit_Click"/></td>
        </tr>
      </table>
    </div>
  </form>
</body>
```

register.aspx.cs:

```
protected void Page_Load(object sender, EventArgs e)
{
    if (!this.IsPostBack)
        txtFname.Focus();
}
protected void btnSubmit_Click(object sender, EventArgs e)
{
    //Store form data in Context.Items
    Context.Items["First Name"] = txtFname.Text;
    Context.Items["Last Name"] = txtLname.Text;
    Context.Items["Email"] = txtEmail.Text;
    Context.Items["Gender"] = rblGender.SelectedValue;
    Context.Items["Course"] = ddlCourse.SelectedValue;

    Server.Transfer("display.aspx");
}
```

display.aspx:

```
<body>
    <form id="form1" runat="server">
        <div>
            <h2>Registered User Information:</h2>

            <asp:Label ID="lblFname" runat="server" /><br /><br />
            <asp:Label ID="lblLname" runat="server" /><br /><br />
            <asp:Label ID="lblEmail" runat="server" /><br /><br />
            <asp:Label ID="lblGender" runat="server" /><br /><br />
            <asp:Label ID="lblCourse" runat="server" /><br /><br />
        </div>
    </form>
</body>
```

display.aspx.cs:

```
protected void Page_Load(object sender, EventArgs e)
{
    //Read the data passed via Context.Items
    lblFname.Text = "First Name: " + (string)Context.Items["First Name"];
    lblLname.Text = "Last Name: " + (string)Context.Items["Last Name"];
    lblEmail.Text = "Email: " + (string)Context.Items["Email"];
    lblGender.Text = "Gender: " + (string)Context.Items["Gender"];
    lblCourse.Text = "Course: " + (string)Context.Items["Course"];
}
```

Outputs:

Registration Form

Enter your first name:

Enter your last name:

Enter your email id:

Enter your gender: ☒ Male ☐ Female

Select a course:

Registered User Information:

First Name: Diwas

Last Name: Lamichhane

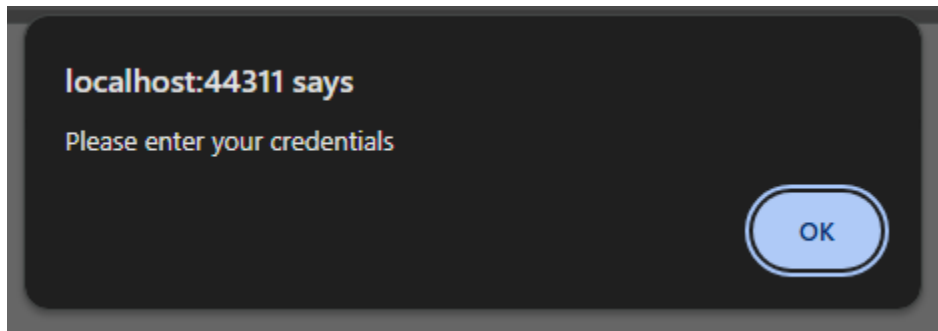
Email: lamichhanediwas7@gmail.com

Gender: Male

Course: BCA

Note: You can also add validations like this in your register.aspx.cs page

```
if (string.IsNullOrEmpty(txtFname.Text) || string.IsNullOrEmpty(txtLname.Text) ||
    string.IsNullOrEmpty(txtEmail.Text) || rblGender.SelectedIndex == -1 ||
    ddlCourse.SelectedIndex == -1 )
{
    Response.Write("<script>alert('Please enter your credentials');</script>");
    return;
}
```



#TU (2022) 17. Write a program to create form for calculating simple interest in one ASP.NET page and display the simple interest in another page of ASP.NET.

simpleInterestCalc.aspx

```
<body>
<form id="form1" runat="server">
    <div>
        <table align="center">
            <caption>Simple Interest Calculator</caption>
            <tr>
                <td>Enter Principal (P) (in Rs) :</td>
                <td><asp:TextBox ID="txtPrincipal" runat="server"></asp:TextBox></td>
            </tr>
            <tr>
                <td>Enter Time (T) (in years) :</td>
                <td><asp:TextBox ID="txtTime" runat="server"></asp:TextBox></td>
            </tr>
            <tr>
                <td>Enter Rate (R) (in % pa):</td>
                <td><asp:TextBox ID="txtRate" runat="server"></asp:TextBox></td>
            </tr>
            <tr>
                <td align="right"><asp:Button ID="btnCalc" runat="server" Text="Calculate"
                OnClick="btnCalc_Click" /></td>
            </tr>
        </table>
    </div>
```



```
</form>
</body>
```

simpleInterestCalc.aspx.cs:

```
protected void Page_Load(object sender, EventArgs e)
{
    if (!this.IsPostBack)
        txtPrincipal.Focus();
}
protected void btnCalc_Click(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(txtPrincipal.Text) || string.IsNullOrEmpty(txtTime.Text)
        || string.IsNullOrEmpty(txtRate.Text))
    {
        Response.Write("<script>alert('Please enter all fields.');
```

SIdisplay.aspx:

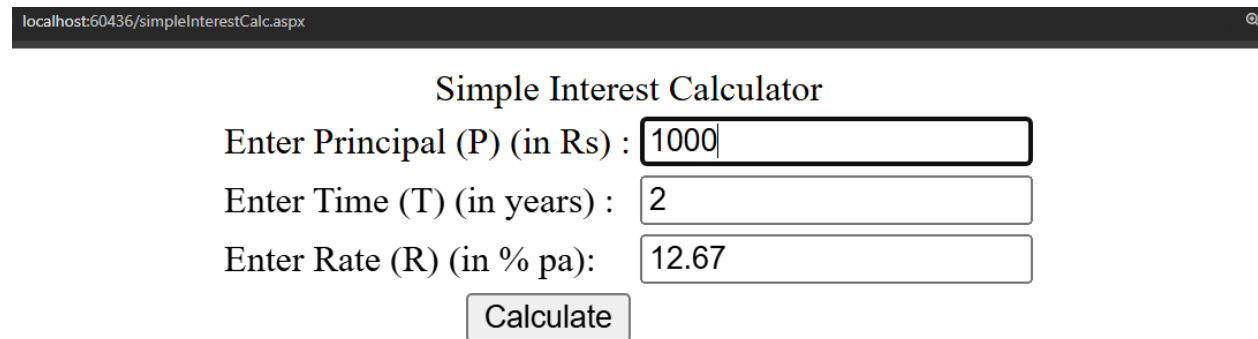
```
<body>
    <form id="form1" runat="server">
        <h2>Result of Simple Interest:</h2>

        <asp:Label ID="lblPrincipal" runat="server"/><br />
        <asp:Label ID="lblTime" runat="server"/><br />
        <asp:Label ID="lblRate" runat="server"/><br />
        <asp:Label ID="lblSI" runat="server"/><br />
    </form>
</body>
```

SIDisplay.aspx.cs:

```
protected void Page_Load(object sender, EventArgs e)
{
    lblPrincipal.Text = "Principal: Rs. " + Context.Items["Principal"];
    lblTime.Text = "Time: " + Context.Items["Time"] + " years";
    lblRate.Text = "Rate: " + Context.Items["Rate"] + " %";
    lblSI.Text = "Simple Interest: Rs. " + Context.Items["Simple Interest"];
}
```

Output:



The screenshot shows a web browser window with the address bar displaying 'localhost:60436/simpleInterestCalc.aspx'. The page content is titled 'Simple Interest Calculator'. It contains three input fields with labels: 'Enter Principal (P) (in Rs) :', 'Enter Time (T) (in years) :', and 'Enter Rate (R) (in % pa):'. The values entered are 1000, 2, and 12.67 respectively. Below these fields is a 'Calculate' button.

Simple Interest Calculator	
Enter Principal (P) (in Rs) :	1000
Enter Time (T) (in years) :	2
Enter Rate (R) (in % pa):	12.67
Calculate	

Result of Simple Interest:

Principal: Rs. 1000

Time: 2 years

Rate: 12.67 %

Simple Interest: Rs. 253.4

TU (2020) 18 b) : Write a program for handling exception in ASP.NET
exceptionDemo.aspx:

```
<body>
  <form id="form1" runat="server">
    <table align="center">
      <caption>Division of Two Numbers</caption>
      <tr>
        <td>Enter 1st number:</td>
        <td><asp:TextBox ID="txtNum1" runat="server"></asp:TextBox></td>
      </tr>
      <tr>
        <td>Enter 2nd number:</td>
        <td><asp:TextBox ID="txtNum2" runat="server"></asp:TextBox></td>
      </tr>
      <tr>
        <td align="right"><asp:Button ID="btndivide" runat="server"
OnClick="btndivide_Click" /></td>
        <td align="right" Text="Divide(/)"></td>
      </tr>
      <tr>
        <td align="center"><asp:Label ID="lblResult" runat="server" ForeColor="Green"></asp:Label>
        <asp:Label ID="lblError" runat="server" ForeColor="Red"></asp:Label>
      </td>
      <td align="center"></td>
      </tr>
    </table>
  </form>
</body>
```

exceptionDemo.aspx.cs:

```
protected void Page_Load(object sender, EventArgs e)
{
    if (!this.IsPostBack)
        txtNum1.Focus();
}
protected void btnDivide_Click(object sender, EventArgs e)
{
    lblResult.Text = "";
    lblError.Text = "";

    try
    {
        int num1 = Convert.ToInt32(txtNum1.Text);
        int num2 = Convert.ToInt32(txtNum2.Text);
        int result = (num1 / num2);

        lblResult.Text = "Result: " + num1 + "/" + num2 + "=" + result;
    }
    catch (DivideByZeroException)
    {
        lblError.Text = "Can't divide by zero";
    }
    catch (FormatException)
    {
        lblError.Text = "Please enter valid whole numbers";
    }
    catch (Exception ex)
    {
        lblError.Text = "Sorry, something went wrong";
    }
}
```

Output:

Can't divide by zero

Division of Two Numbers

Enter 1st number:	14
Enter 2nd number:	0
Divide(/)	

Please enter valid whole numbers

Division of Two Numbers

Enter 1st number:

Enter 2nd number:

Result: $14/2=7$

Division of Two Numbers

Enter 1st number:

Enter 2nd number:

Sorry, something went wrong

Division of Two Numbers

Enter 1st number:

Enter 2nd number: